AI-driven Software Engineering

Assignment #04


WeLcome

Agenda: Assignment #04 - AI-driven Software Engineering

1. Assignment #04
2. References
3. Submission details
4. Q & A

Agenda: Assignment #04 - AI-driven Software Engineering

1. **Assignment #04**
2. References
3. Submission details
4. Q & A

Assignment details

- AI-assisted software evolution, refactoring and quality improvement.
- Scenario: Smart Campus Room Availability Checker.
- Work individually. Extend a working application rather than repeat the Practice #08 debugging exercise.
- Use at least one AI assistant; independently verify its recommendations

Learning outcomes

- Onboard to an unfamiliar or AI-generated codebase using structured context.

- Extend functionality without breaking existing acceptance criteria.

continues …

Learning outcomes

- Refactor code into maintainable components.
- Design and execute automated regression tests.
- Evaluate software quality with evidence, not impressions.

What is different from Practice #08?

- Practice #08: reproduce defects, create SKILL.md manually, and compare debugging with and without the skill.

- Assignment #04: implement NEW product requirements, reorganize the codebase, preserve old behavior, and evaluate engineering quality.

- Creating a new Agent Skill is NOT required.

Starting point and scope

- Continue your Assignment #03 / Practice #06 Smart Campus prototype, or reconstruct an equivalent minimal version.

- One room availability checker with fictional bookings and a fixed current time.

continues …

Starting point and scope

- Preserve AC1–AC6: valid time, 120-minute maximum, blocked room, overlap rules, immutable booking data, clear responsive feedback.

- No login, payments, live APIs or database.

New feature: room selection

- Extend the prototype from one room to a small fictional catalog of at least FOUR rooms.
- Each room must have an ID, name, capacity, equipment list, blocked status and its own bookings.
- Display only rooms satisfying the user’s selected requirements.
- The availability check must use the chosen room bookings, not another room’s data.

New feature: filtering rules

F1. Capacity filter: room capacity must be greater than or equal to requested attendees.

F2. Equipment filter: room must contain ALL selected required items.

F3. A blocked room cannot be reported as available.



continues …

New feature: filtering rules

F4. Show a clear empty state when no rooms match.

F5. Changing the selected room must refresh the availability result appropriately.



Task #01 - Codebase analysis

- Give your AI assistant a concise repository map, acceptance criteria and project constraints.
- Identify at least THREE maintainability risks, supported by file/function references.
- Propose a prioritized improvement plan before editing code.
- Explain one AI suggestion you rejected or modified.

Task #02 - Feature extension

- Add room catalog and filtering by capacity and equipment.
- Preserve original time and overlap validation.
- Use fictional data only; keep the interface responsive.
- Demonstrate successful and unsuccessful filtering, room switching, and availability checks.

Task #03 - Refactoring

- Separate room data, booking/availability rules, filtering logic and UI presentation.
- Reduce duplicated validation or rendering logic.
- Keep behavior consistent before and after refactoring.
- Supply a before/after module or component diagram and justify at least TWO design decisions.

Task #04 - Automated testing

- Run at least TEN executable tests.
- Include original AC1–AC5 regression checks and new F1–F5 tests.
- Cover boundaries: exactly 120 vs 121 minutes; touching vs overlapping intervals; exact capacity vs insufficient capacity.
- Add at least ONE integration or UI behavior check.
- Record actual commands and pass/fail results.

Task #05 - Quality evaluation

Compare original and final versions using a short evidence table:

- Automated tests passed / total
- Number of duplicated logic locations removed
- Code structure: modules/components and responsibilities
- At least one remaining limitation

Do not invent metrics; explain how each measure was obtained.

Task #06 - AI reflection

Write 200–300 words addressing:

- What context did you provide the AI assistant?
- Which generated suggestions were useful or incorrect?
- What did you personally verify?
- How did tests influence your engineering decisions?
- What would you improve next?

Suggested AI workflow

1. Inspect repository and record a baseline.
2. Provide AI with project map and explicit requirements.
3. Ask for a small implementation plan, not a full rewrite.
4. Implement one change at a time.
5. Run tests and review diffs after each change.
6. Document evidence and unresolved limitations.

Optional advanced extension

- Use an existing Agent Skill or an approved MCP-enabled development environment to assist codebase navigation or review.

- Do not create another SKILL.md for this assignment.
- Keep tool permissions minimal; do not share credentials or private data.
- This extension is optional and does not replace the six graded tasks.

Agenda: Assignment #04 - AI-driven Software Engineering

1. Assignment #04
2. **References**
3. Submission details
4. Q & A

References details

- Lesson #07 — Lecture: Advanced Context Engineering.
- Lesson #07 — Practice: Testing AI-Generated Code.

- Lesson #08 — Lecture: Agent Skills and CLI.
- Lesson #08 — Practice: Fix and improve a buggy codebase.

- Assignment #03 — Smart Campus prototype baseline.

Agenda: Assignment #04 - AI-driven Software Engineering

1. Assignment #04
2. References
3. **Submission details**
4. Q & A

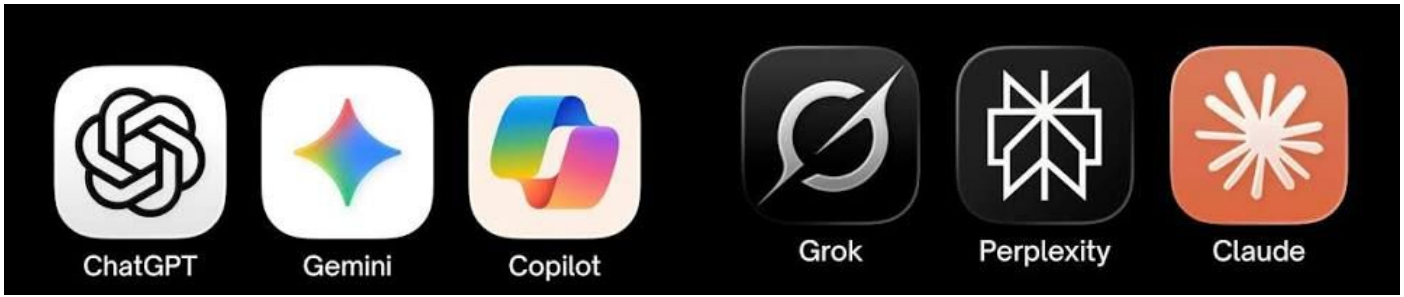
Submission details

AI assistant indication

Please indicate AI assistant name you have used for assignment work

AI-use record

- Include the tool and date, prompt summary, output used, problems found, changes made, and verification sources.



Required deliverables

- Source code for the initial and final versions (or Git commit references).
- README with setup, run and test commands.
- Test files and actual execution results.
- Brief analysis/refactoring report with diagrams and quality comparison.
- AI evidence appendix: tool/model, selected prompts, outputs, decisions and verification.

Submission checklist

- Existing availability rules still work.
- At least four fictional rooms are included.
- Capacity and equipment filters work.
- At least ten executable tests are provided.
- Refactoring and evaluation evidence are included.
- AI use and human review are documented.

Submission details

Please send to *SIS #04* in ***AI-driven Software Engineering - Fall 2026 [WED18-21]*** class



Submission details

DEADLINE: November 10, 2026 23:59:59



Submission details

Submissions that do not adhere to the required formatting guidelines will be rejected, regardless of whether the work itself is fully completed. Please note that resubmission will not be permitted if the incorrect format is used.

Assignment work must be completed in its entirety, as the opportunity to resubmit will not be provided if it remains incomplete.



Agenda: Assignment #04 - AI-driven Software Engineering

Q & A